Smart Water Testing Document

Introduction

This testing document outlines the approach, methodologies, test types, and test scenarios for verifying the functionality, performance, and security of the SmartWaterMeter system. The SmartWaterMeter project uses fully homomorphic encryption via Microsoft SEAL to enable secure and privacy-preserving computations on water consumption data. The primary goal of the test plan is to ensure that the cryptographic operations, core functionalities, integration points, performance parameters, and security assumptions hold true under various conditions.

Objectives

1. Functional Validation: Confirm that the system's features, such as encryption, decryption, homomorphic addition / multiplication, and data aggregation work as intended.
2. Correctness of Cryptographic Operations: Validate that the Microsoft SEAL-based encryption and homomorphic operations produce correct results, ensuring ciphertexts remain valid and decrypt properly.
3. Performance and Scalability: Measure and ensure acceptable computation times, memory usage, and scalability under increasing load and data volumes.
4. Robustness and Stability: Test the system's behavior under edge cases, invalid inputs, and error conditions to ensure it degrades gracefully or provides meaningful error messages.
5. Security Assumptions: Confirm that no plaintext data is leaked and that operations maintain the intended security properties of the homomorphic encryption scheme.

Test Environment

The hardware / operating system required can be any modern system, provided that it has at least 8 GB of ram, C++ installed past 9.3.0 or clang++ with C++17 support, and CMake 3.10 for build configuration. The software required is Microsoft SEAL, CMake, OpenMP, GoogleTest (optional), and python/bash for automated test orchestration (optional).

Test Cases

1. Unit Tests

Test ID	Module	Description	Input	Result
1	**KeyGen**	**Verify key generation produces valid keys**	**None**	**Keys are generated without error**

2	**Encrypt**	**Encrypts a single integer plaintext**	**Plaintext: 42**	**Ciphertext decrypts back to 42**
3	**Decrypt**	**Decrypt a known ciphertext**	**Ciphertext: 42**	**Correct plaintext (42)**
4	**Add**	**Add two ciphertexts homomorphically**	**Ciphertexts: (42, 58)**	**Decrypt result: 100**
5	**Mul**	**Multiply two ciphertexts homomorphically**	**Ciphertexts: (6, 7)**	**Decrypt result: 42**
6	**InvalidInput**	**Attempt to decrypt malformed ciphertext**	**Corrupted Ciphertext**	**Graceful error handling (exception)**

2. **Integration Tests**

Test ID	Scenario	Steps	Result
1	**Single Meter Calculation**	**1. Generate Keys. 2. Encrypt daily usage 3. Hom-add up weeks usage 4. Decrypt Result**	**Correct total usage matches plaintext sum**

2	**Multiple Meters Aggregation**	**1. Generate keys 2. Encrypt multiple meters data 3. Hom-add across all meters 4. Decrypt result**	**Result matches sum of all meters data**
3	**Mixed Operations**	**1. Generate keys 2. Encrypt daily usage 3. Perform a sequence of additions and multiplications 4. Decrypt results**	**Result matches expected mathematical outcomes**

Testing Procedures

- Unit Tests: ./build/tests/unit_test.sh
- Integration Tests: ./build/tests/integration_test.sh
- Performance Tests: ./build/tests/performance_test.sh
- Regression Tests: ./build/tests/regression_test.sh

Pass/Fail Criteria

- Function Tests: All core functions (encryption, decryption, homomorphic operations) return correct results for known inputs.
- Performance Tests: The system completes operations within defined performance thresholds
- Security Tests: No plaintext or key leakage is observed in memory, logs, or error messages. Tampered data is handled gracefully without compromising keys.

Risks and Mitigations

1. Risk: Incompatibility of Microsoft SEAL versions or build failures due to dependency issues.
    - Mitigation: Follow the exact version and installation instructions from the repositories documentation. Use CI with a known working environment.
2. Risk: Slow performance when scaling to large data sets.
    - Mitigation: Identify and profile bottlenecks, optimize parameter selection for SEAL (e.g. polynomial degree, security level), and adjust configuration.
3. Risk: Insufficient handling of invalid or corrupted ciphertexts.
    - Test extensively with corrupted inputs and ensure exception handling is robust.